Libtrace User Guide

Gustavo Bacagine
2023 - 2024

Libtrace User Guide

Gustavo Bacagine
2023 - 2024

Table of Contents

DOCUMENT INFORMATIONS....5
1 INTRODUCTION....6
1.1 UNDERSTANTING THE PROGRAMMING METHOD USED IN THIS FILE....6
1.1.1 Hungarian Notation Usage Example....6
2 DEFINES AND MACROS....8
2.1 DEBUG LEVELS....8
2.2 FUNCTIONS MACROS....8
3 GLOBAL VARIABLES....9
4 FUNCTIONS....10
4.1 SETTERS....10
4.2 GETTERS....10
4.3 PRINT TRACE MESSAGE....10
APENDIX A – SOURCE CODE....11
trace.h....11
trace.c....12
APENDIX B – USAGE EXAMPLES....14
Example 1 - debug.c....14
Example 2 - hello_trace.c....16
Example 3 - trace_function.c....16
Example 4 - trace_level.c....17
Example 5 - Modular programming....18
module1.c:....18
module2.c:....19

DOCUMENT INFORMATIONS

Author	Gustavo Bacagine
Version	1.1
Document Date	15/Apr/2024

1 INTRODUCTION

The libtrace is a C trace library developed by Gustavo Bacagine <gustavo.bacagine@protonmail.com>.

This library is a free software

1.1 UNDERSTANTING THE PROGRAMMING METHOD USED IN THIS FILE

In my programs I use a customized version of Hungarian Notation.

Hungarian notationaims to facilitate recognition of the type of variable in a program, just by looking at the name of the variable. See below a table with the type of indicators for each type of data.

Indicator	Type
g	Global
k	Constant
a	Array
p	Pointer
v	Void
b	Boolean
ch	Char
i	Integer
f	Fload
d	Double
sz	String
st	Struct
un	Union
e	Enum
fp	File Pointer
Dp	Directory Pointer

1.1.1 Hungarian Notation Usage Example

```
typedef struct STRUCT_PERSON {
  int iID;
  char szName[64];
  char cSex;
  float fHeight;
  double dSalary;
  bool bLive;
```

```
} STRUCT_PERSON, *PSTRUCT_PERSON

const double gkdPI = 3.14;

...

int main(int argc, char **argv) {
  PSTRUCT_PERSON pstPerson = (PSTRUCT_PERSON) malloc(sizeof(STRUCT_PERSON);

  ...

  free(pstPerson);
  pstPerson = NULL;

  return 0;
}
```

2 DEFINES AND MACROS

In this chapter I explain the defines and macros defined in trace.h

2.1 DEBUG LEVELS

```
#define INFO_LEVEL    0
#define WARNING_LEVEL 1
#define ERROR_LEVEL   2
#define FATAL_LEVEL   3
#define DEBUG_LEVEL   4
#define TRACE_LEVEL   5
```

INFO_LEVEL (0): Normal messages

WARNING_LEVEL (1): Alert messages

ERROR_LEVEL (2): Errors that not cause the interruption of program

FATAL_LEVEL (3): Erros that cause the interruption of program

DEBUG_LEVEL (4): Detailed messages intended for the programmer

TRACE_LEVEL (5): Messages more detailed than DEBUG_LEVEL level. Show all trace level messages

2.2 FUNCTIONS MACROS

```
#define vTrace(FORMAT, ...) _vTrace(__FILE__, __LINE__, FORMAT, ##__VA_ARGS__)

#define vTraceBegin() _vTrace(__FILE__, __LINE__, "%s - begin", __func__)
#define vTraceEnd()   _vTrace(__FILE__, __LINE__, "%s - end", __func__)
```

The first define is used to print a message in a .log file.

The vTraceBegin is used in the beginning of a function, and vTraceEnd is used in the end of a funciton.

Example:

```
void vFunc(void) {
  vTraceBegin();

  if(INFO_DETAILS) vTrace("Hello World!!!");

  vTraceEnd();
}
```

3 GLOBAL VARIABLES

```
extern char gszTrace[trace_FILE_NAME_LENGTH];
```

Used to save the name of .log file

```
extern int giDebugLevel;
```

Used to save the debug level

4 FUNCTIONS

4.1 SETTERS

```
void vSetTraceFileName(const char *ksztraceFileName);
```

Set the name of .log file

```
void vSetDebugLevel(const int kiDebugLevel);
```

Set the value of `giDebugLevel` variable

4.2 GETTERS

```
int iGettraceLevel(void);
```

Get the value of `giDebugLevel` variable

4.3 PRINT TRACE MESSAGE

```
void _vTrace(const char *kpszModuleName,
             const int kiLine,
             const char *kpszFmt, ...);
```

Print a message in the .log file

APENDIX A – SOURCE CODE

trace.h

```
/**
 * trace.h: This file contains constants and functions to create
 * .log files
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmai.com>
 *
 * Date: 2023-06-16
 */

#ifndef _TRACE_H_
#define _TRACE_H_

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <stdarg.h>
#include <time.h>
#include <errno.h>

#ifdef LINUX
  #include <sys/time.h>
  #include <unistd.h>
#else
  #include <windows.h>
#endif

#define INFO_LEVEL    1 // Normal messages
#define WARNING_LEVEL 2 // Warning alert messages
#define ERROR_LEVEL   3 // Error messages
#define FATAL_LEVEL   4 // Fatal error messages
#define DEBUG_LEVEL   5 // Messages with more information about the source code
#define TRACE_LEVEL   6 // Show all messages in trace

/**
 * Example of use:
 *
 * if(INFO_DETAILS) {
 *   vTraceInfo("INFO Message");
 * }
 *
 * if(DEBUG_DETAILS) {
 *   vTraceDebug("DEBUG Message");
 * }
 */
#define INFO_DETAILS    giDebugLevel >= INFO_LEVEL
#define WARNING_DETAILS giDebugLevel >= WARNING_LEVEL
#define ERROR_DETAILS   giDebugLevel >= ERROR_LEVEL
#define FATAL_DETAILS   giDebugLevel >= FATAL_LEVEL
#define DEBUG_DETAILS   giDebugLevel >= DEBUG_LEVEL
#define TRACE_DETAILS   giDebugLevel >= TRACE_LEVEL

#define vTrace(FORMAT, ...) _vTrace(__FILE__, __LINE__, FORMAT, ##__VA_ARGS__)

#define vTraceBegin() _vTrace(__FILE__, __LINE__, "%s - begin", __func__)
#define vTraceEnd()   _vTrace(__FILE__, __LINE__, "%s - end", __func__)

extern char gszTrace[256];
```

```
extern int giDebugLevel;

void vSetTraceFileName(const char *kszLogFileName);
void vSetDebugLevel(const int kiDebugLevel);
int iGetDebugLevel(void);

void _vTrace(const char *kpszModuleName,
             const int kiLine,
             const char *kpszFmt, ...);

#endif
```

trace.c

```
/**
 * trace.c: This file contains the implementation of traceger functions
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmail.com>
 *
 * Date: 2023-06-16
 */

#include "trace.h"

char gszTrace[256] = "";
int giDebugLevel = 1;

void vSetTraceFileName(const char *kszLogFileName) {
  snprintf(gszTrace, sizeof(gszTrace), "%s", kszLogFileName);
}

void vSetDebugLevel(const int kiDebugLevel) {
  giDebugLevel = kiDebugLevel;
}

int iGetDebugLevel(void) {
  return giDebugLevel;
}

void _vTrace(const char *kpszModuleName,
             const int kiLine,
             const char *kpszFmt, ...) {
  va_list args;
  FILE *fpLogFile = NULL;
  time_t tTime = time(NULL);
  struct timeval stTimeVal;
  struct tm *stTime = localtime(&tTime);
  char *pszDbg = NULL;
  pid_t iPID = getpid();
  int iReallocBuffer = 0;

  gettimeofday(&stTimeVal, NULL);
  stTime = localtime(&stTimeVal.tv_sec);

  if ( ( fpLogFile = fopen(gszTrace, "a") ) == NULL ) {
    fprintf(stderr, "E: Impossible create or open file %s!\n"
                    "%s\n", gszTrace, strerror(errno));
    exit(EXIT_FAILURE);
  }

  va_start(args, kpszFmt);

  iReallocBuffer = snprintf(NULL, 0,
```

```
    "%02d/%02d/%04d %02d:%02d:%02d:%03ld %d %s:%d %s\n",
    stTime->tm_mday,
    stTime->tm_mon + 1,
    stTime->tm_year + 1900,
    stTime->tm_hour,
    stTime->tm_min,
    stTime->tm_sec,
    stTimeVal.tv_usec / 1000,
    iPID,
    kpszModuleName,
    kiLine,
    kpszFmt
  );

  pszDbg = (char *) malloc(sizeof(char) * (iReallocBuffer+1));

  if ( pszDbg == NULL ) {
    fprintf(stderr, "E: Out memory!\n");
    exit(EXIT_FAILURE);
  }

  memset(pszDbg , 0x00, iReallocBuffer+1);

  iReallocBuffer = snprintf(pszDbg, iReallocBuffer+1,
    "%02d/%02d/%04d %02d:%02d:%02d:%03ld %d %s:%d %s\n",
    stTime->tm_mday,
    stTime->tm_mon + 1,
    stTime->tm_year + 1900,
    stTime->tm_hour,
    stTime->tm_min,
    stTime->tm_sec,
    stTimeVal.tv_usec / 1000,
    iPID,
    kpszModuleName,
    kiLine,
    kpszFmt
  );

  vfprintf(fpLogFile, pszDbg, args);
  fflush(fpLogFile);

  free(pszDbg);
  pszDbg = NULL;

  fclose(fpLogFile);
  fpLogFile = NULL;

  va_end(args);
}
```

APENDIX B – USAGE EXAMPLES

Example 1 - debug.c

```
/**
 * debug.c: Exemple of the DEBUG_DETAILS
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmail.com>
 *
 * Date: 2023-06-17
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include "trace.h"

#define UNUSED(X) (void) X

typedef struct STRUCT_PERSON {
  char szName[64];
  uint8_t uAge;
} STRUCT_PERSON, *PSTRUCT_PERSON;

static void vInitSTRUCT_PERSON(PSTRUCT_PERSON *ppstPerson) {
  if ( INFO_DETAILS ) {
    vTrace("begin %s function", __func__);
    vTrace("Setting the default values in members of struct STRUCT_PERSON");
  }

  memset((*ppstPerson), 0x00, sizeof(PSTRUCT_PERSON *));

  if ( DEBUG_DETAILS ) {
    vTrace("memset((*ppstPerson), 0x00, sizeof(PSTRUCT_PERSON *))");
  }

  if ( INFO_DETAILS ) {
    vTrace("end %s function", __func__);
  }
}

static void vCreateSTRUCT_PERSON(PSTRUCT_PERSON *ppstPerson, const char *szName,
const uint8_t uAge) {
  if ( INFO_DETAILS ) {
    vTrace("begin %s function", __func__);
    vTrace("create a new person");
  }

  strcpy((*ppstPerson)->szName, szName);
  (*ppstPerson)->uAge = uAge;

  if ( DEBUG_DETAILS ) {
    vTrace("Values of the member STRUCT_PERSON");
    vTrace("(*ppstPerson)->szName = %s", (*ppstPerson)->szName);
    vTrace("(*ppstPerson)->uAge = %d", (*ppstPerson)->uAge);
  }

  if ( INFO_DETAILS ) {
    vTrace("STRUCT_PERSON created with succes");
    vTrace("end %s function", __func__);
```

```
  }
}

static void vShowSTRUCT_PERSON(PSTRUCT_PERSON pstPerson) {
  if ( INFO_DETAILS ) {
    vTrace("begin %s function", __func__);
    vTrace("Show STRUCT_PERSON");
    vTrace("Name: %s\n"
           "Age.: %d\n", pstPerson->szName,
                         pstPerson->uAge);
  }

  printf("Name: %s\n"
         "Age.: %d\n", pstPerson->szName,
                       pstPerson->uAge);

  if ( INFO_DETAILS ) vTraceEnd();
}

static void vDestroyPerson(PSTRUCT_PERSON *ppstPerson) {
  if ( INFO_DETAILS ) vTraceBegin();

  memset((*ppstPerson), 0x00, sizeof(PSTRUCT_PERSON *));

  if ( DEBUG_DETAILS ) {
    vTrace("memset((*ppstPerson), 0x00, sizeof(PSTRUCT_PERSON *))");
    vTrace("Freeing STRUCT_PERSON structure pointer");
  }

  free((*ppstPerson));
  (*ppstPerson) = NULL;

  if ( DEBUG_DETAILS ) {
    vTrace("free((*ppstPerson))");
    vTrace("(*ppstPerson) = NULL");
    vTrace("STRUCT_PERSON structure pointer free");
  }

  if ( INFO_DETAILS ) vTraceEnd();
}

int main(int argc, char **argv) {
  PSTRUCT_PERSON pstPerson = (PSTRUCT_PERSON) malloc(sizeof(STRUCT_PERSON));

  UNUSED(argc);
  UNUSED(argv);

  vSetTraceFileName("debug.log");
  vSetDebugLevel(DEBUG_LEVEL);

  // Initial trace messages
  if ( INFO_DETAILS ) {
    vTrace("begin of function %s", __func__);
  }

  if ( pstPerson == NULL ) {
    if ( DEBUG_DETAILS ) {
      vTrace("E: out memory in allocation of pstPerson!");
    }
  }

  vInitSTRUCT_PERSON(&pstPerson);
```

```
  vCreateSTRUCT_PERSON(&pstPerson, "Gustavo Bacagine", 23);

  vShowSTRUCT_PERSON(pstPerson);

  vDestroyPerson(&pstPerson);

  // End trace messages
  if ( INFO_DETAILS ) {
    vTrace("end of function %s", __func__);
  }

  return 0;
}
```

Example 2 - hello_trace.c

```
/**
 * hello_trace.c
 *
 * Writeen by Gustavo Bacagine <gustavo.bacaigne@protonmail.com>
 *
 * Descritpion: Print a hello world in a trace file.
 *
 * Date: 23/09/2023
 */

#include <stdio.h>
#include "trace.h"

#define UNUSED(X) (void) X

int main(int argc, char **argv) {
  UNUSED(argc);
  UNUSED(argv);

  vSetDebugLevel(INFO_DETAILS);

  if ( giDebugLevel < 1 ) {
    fprintf(stderr, "Error, giDebugLevel return value: %d!\n", giDebugLevel);
    exit(EXIT_FAILURE);
  }

  vSetTraceFileName("hello_trace.log");

  if ( INFO_DETAILS ) {
    vTrace("%s - begin", __func__);
    vTrace("Hello World!!!");
    vTrace("%s - end", __func__);
  }

  return 0;
}
```

Example 3 - trace_function.c

```
/**
 * trace_function.c
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmail.com>
 *
```

```
 * Description: File created to test the new macros:
 *  vTrace
 *  vTraceBegin
 *  vTraceEnd
 *
 * Date: 2023-10-17
 *
 */

#include <stdio.h>
#include "trace.h"

#define UNUSED(X) (void) X

void vMyFunc(void) {
  if ( INFO_DETAILS ) vTraceBegin();

  if ( INFO_DETAILS ) vTrace("INFO MESSAGE");
  if ( WARNING_DETAILS ) vTrace("WARNING MESSAGE");
  if ( ERROR_DETAILS ) vTrace("ERROR MESSAGE");
  if ( FATAL_DETAILS ) vTrace("FATAL MESSAGE");
  if ( DEBUG_DETAILS ) vTrace("DEBUG MESSAGE");
  if ( TRACE_DETAILS ) vTrace("TRACE MESSAGE");

  if ( INFO_DETAILS ) vTraceEnd();
}

int main(int argc, char **argv) {
  UNUSED(argc);
  UNUSED(argv);
  UNUSED(gszTrace);

  vSetDebugLevel(TRACE_LEVEL);

  vSetTraceFileName("trace.log");

  if ( INFO_DETAILS ) {
    vTraceBegin();
    vTrace("Main Info Message :)");
  }

  vMyFunc();

  if ( INFO_DETAILS ) vTraceEnd();

  return 0;
}
```

Example 4 - trace_level.c

```
/**
 * trace_level.c: Test to trace library with TRACE_DETAILS
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmail.com>
 *
 * Date: 2023-06-16
 */

#include <stdio.h>
#include "trace.h"

#define UNUSED(X) (void) X
```

```
int main(int argc, char **argv, char **envp) {
  int ii;

  UNUSED(gszTrace);

  vSetDebugLevel(TRACE_LEVEL);

  if ( giDebugLevel < 1 ) {
    fprintf(stderr, "Error, giDebugLevel return value: %d!\n", giDebugLevel);
    exit(EXIT_FAILURE);
  }

  vSetTraceFileName("trace_level.log");

  if ( INFO_DETAILS ) {
    vTrace("begin %s function", __func__);
    vTrace("INFO Message"      );
  }

  if ( WARNING_DETAILS ) vTrace("WARNING Message");
  if ( ERROR_DETAILS   ) vTrace("ERROR Message"    );
  if ( FATAL_DETAILS   ) vTrace("FATAL Message"    );

  if ( DEBUG_DETAILS ) {
    vTrace("argc = %d", argc);
    for ( ii = 0; ii < argc; ii++ ) {
      vTrace("argv[%d] = %s", ii, argv[ii]);
    }

    for ( ii = 0; envp[ii] != NULL; ii++ ) {
      vTrace("envp[%d] = %s", ii, envp[ii]);
    }
  }

  if ( TRACE_DETAILS ) {
    vTrace("TRACE Message");
    vTrace("%s(argc=%p, argv=%p, envp=%p)", __func__, &argc, &argv, &envp);
  }

  if ( INFO_DETAILS ) {
    vTrace("end %s function", __func__);
  }

  return 0;
}
```

Example 5 - Modular programming

module1.c:

```
/**
 * module1.c: Test to trace library, using modular programming
 *
 * This file is module 1 of 2
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmail.com>
 *
 * Date: 2023-09-14
 */
```

```
#include <stdio.h>
#include "trace.h"

#define UNUSED(X) (void) X

void vShowDebugLevel(void);
void vShowTraceFileName(void);

int main(int argc, char **argv) {
  UNUSED(argc);
  UNUSED(argv);

  vSetDebugLevel(TRACE_LEVEL);

  if ( giDebugLevel < 1 ) {
    fprintf(stderr, "Error, giDebugLevel return value: %d!\n", giDebugLevel);
    exit(EXIT_FAILURE);
  }

  vSetTraceFileName("module.log");

  if ( INFO_DETAILS ) {
    vTrace("begin %s function", __func__);
    vTrace("Debug Level = %d", giDebugLevel);
    vTrace("Log file: %s", gszTrace);
  }

  vShowDebugLevel();
  vShowTraceFileName();

  if(INFO_DETAILS) vTrace("end %s funciton", __func__);

  return 0;
}
```

module2.c:

```
/**
 * module2.c: Test to trace library, using modular programming
 *
 * This file is module 2 of 2
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmail.com>
 *
 * Date: 2023-09-14
 */

#include "trace.h"

#define UNUSED(X) (void) X

void vShowDebugLevel(void) {
  UNUSED(gszTrace);

  if ( INFO_DETAILS ) {
    vTrace("begin %s function", __func__);
    vTrace("Debug Level = %d", giDebugLevel);
  }

  printf("Debug Level = %d\n", giDebugLevel);
```

```
  if ( INFO_DETAILS ) vTrace("end %s funciton", __func__);
}

void vShowTraceFileName(void) {
  if ( INFO_DETAILS ) {
   vTrace("begin %s function", __func__);
   vTrace("Log file: %s", gszTrace);
  }

  printf("Log file: %s\n", gszTrace);

  if ( INFO_DETAILS ) vTrace("end %s funciton", __func__);
}
```